

SmartExpense Software Architecture and Design Specification

Smart Expense and Budget Management Application

Software Engineering Mini Project | Section 5G | Team 14

Version 1.1 | 4 October 2026 | Draft for review

SRN	Team member
PES2UG24CS427	Sahana Shivakumar Badiger
PES2UG24CS428	Sai Arjun M
PES2UG24CS429	Sai Arya A L
PES2UG24CS430	Sai Sathvik Gullipalli

1 Introduction

This document describes how SmartExpense will be built. It connects the SRS requirements to the application modules, database, main user flows and interfaces so Team 14 can implement and test the system.

The application lets users manage their own expenses and income, categories, monthly and category budgets, dashboard charts and CSV exports. It uses INR and shows budget alerts inside the application. Bank integration, statement import, shared budgets, native mobile apps and email or push alerts are outside the scope.

2 Document overview

The intended readers are Team 14, the course coordinator and evaluators. Section 3 describes the architecture and security. Section 4 explains the data, user flows and APIs. Requirement IDs and test cases come from the SRS v1.0 and Test Plan v1.1.

3 Architecture

Goals and constraints

The design aims for correct totals, private user data and simple expense entry. Dashboard and transaction pages should load within 3 seconds for 90% of requests with 5,000 transactions, and support 20 concurrent users. Saved data must survive a restart. The UI must work in Chrome, Firefox and Edge and at a width of 360 px.

Architecture pattern and technology

We will use a three-layer web application: browser UI, one application server and a relational database. The server contains separate modules for each feature. This keeps development and deployment manageable for four students in one semester.

The proposed stack is Python and Flask with Jinja2, HTML, CSS, JavaScript and Chart.js; SQLAlchemy for data access; SQLite locally and PostgreSQL for deployment. Free hosting provides HTTPS. Users need quick entry and privacy; developers need clear modules; the administrator needs backups and simple deployment through server tools.

Component diagram

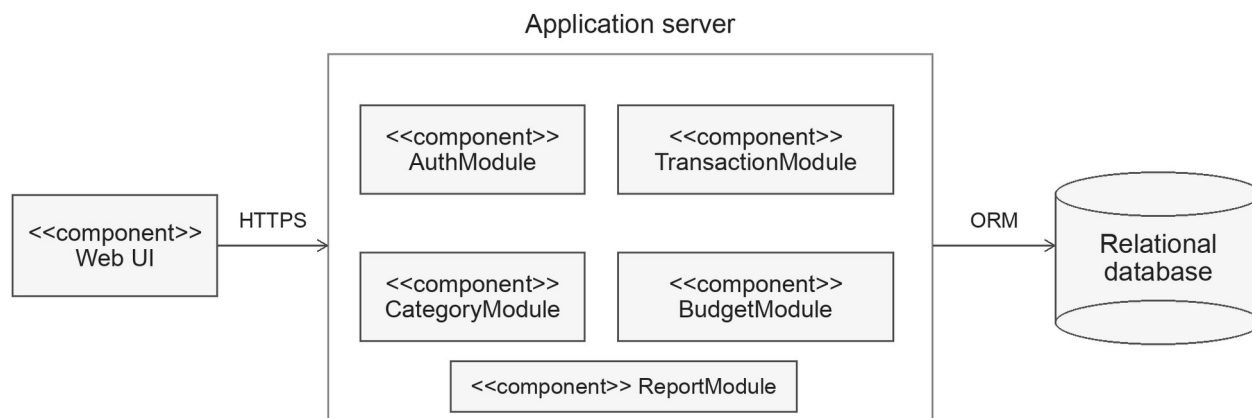


Figure 1 UML component diagram

The browser sends requests to the application server. Each module checks the session and validates input before using the database through SQLAlchemy. The modules run within the same server.

Component descriptions

Component	Responsibility
Web UI	Shows login, dashboard, transaction forms, budgets and categories. Renders charts and errors.
AuthModule	Registers users, checks passwords, creates sessions and handles logout and login lockout.
TransactionModule	Adds, edits, deletes, filters and sorts income and expenses.
CategoryModule	Provides default categories and lets users add or rename their own.
BudgetModule	Stores monthly and category limits and calculates usage and in-app alerts.
ReportModule	Calculates monthly totals, category charts, the six-month trend and filtered CSV exports.
Database	Stores users, sessions, categories, transactions and budgets.

Security architecture

Security protects the confidentiality of user data and prevents changes by anyone except the owner. All protected requests use the logged-in user ID from the server session, including chart and export requests.

- Use HTTPS with TLS 1.2 or later in deployment. Store the session ID in an HttpOnly, Secure cookie and invalidate the session on logout.
- Store only salted bcrypt password hashes. Require at least 8 characters and lock login for 5 minutes after 5 consecutive failures.
- Check ownership of transactions, categories and budgets on every request. Return 404 for another user's record.
- Use ORM queries, escape displayed text and require CSRF tokens for changes. Keep database files and backups outside the public web folder with restricted access.

Risks and mitigations

Slow queries are reduced with indexes on user and date, pagination and database aggregation. Failed writes are rolled back, and backups support recovery. If free hosting is unavailable, local testing can continue while deployment checks wait.

Requirements traceability

Each row links SRS requirements to the design component and existing Test Plan cases. Ranges include every ID between the endpoints. These links describe planned verification; they do not indicate that tests have passed.

SRS requirement IDs	Design component	Test Plan cases
SEB-F-001 to 004	AuthModule and ownership checks	TC-AUTH-01 to 04
SEB-F-005 to 010	TransactionModule	TC-EXP-01 to 06
SEB-F-011	CategoryModule	TC-CAT-01
SEB-F-012 to 014	BudgetModule	TC-BUD-01 to 03
SEB-F-015 to 017	ReportModule and Web UI	TC-DSH-01, TC-DSH-02, TC-RPT-01
SEB-NF-001 to 003	Server queries and database transactions	TC-PERF-01, TC-PERF-02, TC-REL-01
SEB-NF-004 to 005	Web UI and responsive layout	TC-UX-01, TC-PORT-01
SEB-SR-001 to 005	HTTPS, AuthModule, ownership checks and safe input handling	TC-SEC-01 to 05

4 Design

Data design and business rules

Each user owns their transactions, budgets and custom categories. Default categories are shared and read-only. A transaction can refer to a default category or one owned by the same user. The database stores money with two decimal places, and calculations use decimal arithmetic.

Table	Main fields
users	id, name, unique email, password_hash, failed_login_count, locked_until
sessions	id, user_id, expires_at
categories	id, user_id, name; user_id is empty for default categories
transactions	id, user_id, type, amount, date, category_id, payment_mode, source, note
budgets	id, user_id, month, category_id, limit; empty category means monthly budget

- Amounts must be positive and dates cannot be in the future. Expenses require a category and payment mode; income requires a source. Notes are optional.
- Default categories are Food, Travel, Rent, Utilities, Shopping, Health, Education and Other.
- Keep one budget per user, month and category, and one overall budget per month. Budget limits must be positive. Only expenses count towards spending.
- Remaining = budget minus spent. Percentage used = spent divided by budget times 100. If no budget exists, show no usage percentage or threshold alert.
- Show a warning from 80% through 100%, and a distinct over-budget alert above 100%. Alerts never block new entries.
- Recalculate totals and alerts after adding, editing or deleting a transaction. The trend includes the selected month and the previous five months; empty months show zero.

UML sequence diagrams

Flow 1 User login

The normal login flow checks credentials and starts a session. A locked account receives an error until the five-minute lockout expires.

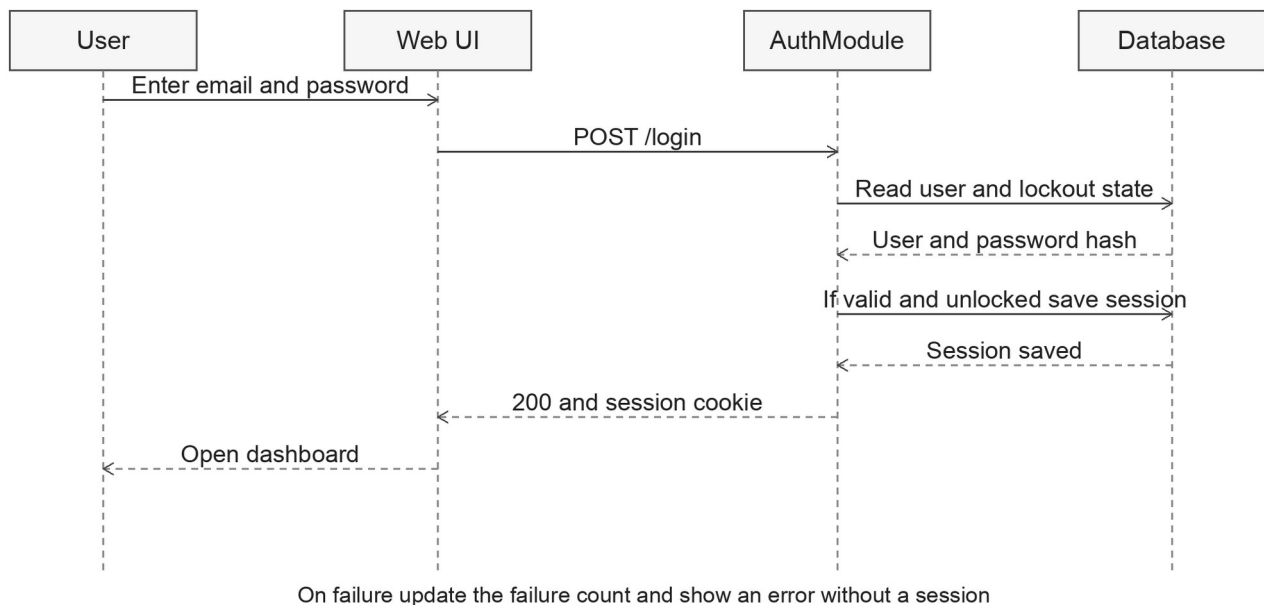
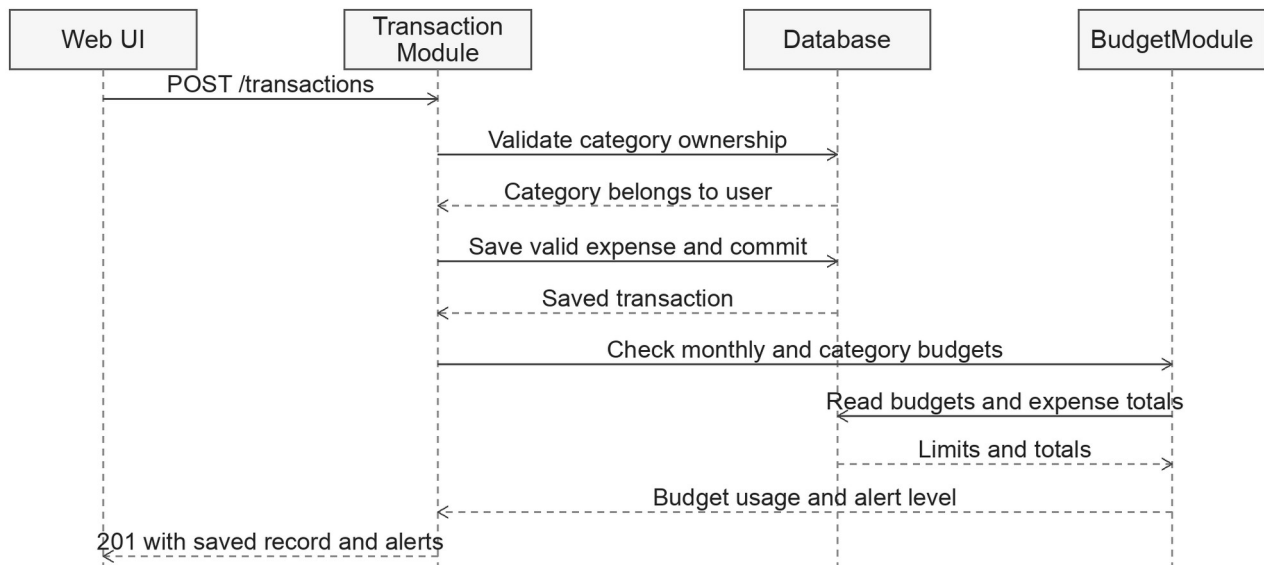


Figure 2 Login sequence for SEB-F-002 and SEB-SR-005

Flow 2 Add an expense

The server checks the session, amount, date and category ownership. After a successful save, it returns updated budget information for the UI to display.



Invalid input returns 422 before saving and an unknown or foreign category returns 404

Figure 3 Expense sequence for SEB-F-005, SEB-F-007 and SEB-F-014

API design

The following interfaces use UTF-8 JSON. Protected routes require a valid session; changes require a CSRF token. Dates use YYYY-MM-DD, months use YYYY-MM and amounts use decimal strings. The server gets the user ID from the session.

Method	Route	Purpose
POST	/register	Create a user account
POST	/login; /logout	Start or end a session
GET, POST	/transactions	List filtered and sorted records; add a record
GET, PUT, DELETE	/transactions/{id}	Read, edit or delete an owned record
GET, POST	/categories	List categories or create a custom category
PUT	/categories/{id}	Rename an owned custom category
GET, PUT	/budgets	Read by month; create or update a budget
GET	/api/dashboard?month=	Totals, budget usage, alerts and chart data
GET	/transactions/export.csv	Export all records matching the list filters

Authentication interface

POST /login request: {"email":"user@example.com", "password":"example123"}

Success: 200 {"message":"Login successful"} and a session cookie.

Errors: 401 for incorrect credentials; 429 during login logout.

Transaction interface

POST /transactions request: {"type":"expense", "amount":"250.00", "date":"2026-10-04", "category_id":1, "payment_mode":"UPI", "note":"Lunch"}

Success: 201 with the saved transaction, budget usage and alerts.

Errors: 401 without a session; 404 for an unknown or foreign category; 422 for invalid input. Income uses a source instead of category and payment mode.

Error handling and logging

Show validation errors beside the field and retain entered values for correction or retry. Return 404 for unavailable records. On a failed save, roll back the write and show a retry message; unexpected failures return 500 without technical details. Log time, route, status and failure type, but never passwords, session tokens or financial notes.

User interface and next steps

Use a consistent navigation menu, labelled forms and a visible Add Expense action. Target expense entry within three clicks. Confirm deletion, show alerts with text and colour, and adapt screens to 360 px width. Implement the modules, run the linked Test Plan cases and update the requirement status after verification. Confirm the proposed stack and hosting before implementation.

5 References and terms

SRS means Software Requirements Specification; UML means Unified Modeling Language; API means Application Programming Interface; ORM maps database records to objects; CSRF means Cross-Site Request Forgery.

References: Team 14 SRS v1.0, 6 September 2026; Test Plan SEB-TP-001 v1.1, 17 September 2026; SE Mini Project Deliverables Part 1; SAD Template and Team 14 SAD reference.